CS315 · ALGORITHMS ANALYSIS & DESIGN

Midterm 442 Solutions

12/01/2023 · 1.5 hours · 25 marks

Muharib

01 Solution Q2: fill in + True / False

Blank	Answer	Reason
InsertItem to dictionary	`O(1)`	From Lecture 3: in a log file we insert at the beginning or the end directly.
Bottom-up heap construction	`O(n)`	Linear in the number of items.
key(u) ≤ __ ≤ key(w)	`key(v)`	The binary search tree property.
Space of a queue (singly linked list)	`O(n)`	One node per element.
FindElement in binary tree	`O(h)`	`h` is the height of the tree: `O(log n)` when balanced and `O(n)` in the worst case.

#	Answer	Reason
1	**F**	The same environment and the same inputs must be used.
2	**T**	The words of the Lecture 1 slide.
3	**T**	We count the maximum number of operations, which is the worst case.

02 Solution Q3: heaps + hash function

	UpHeap	DownHeap
Objective	Restores the heap-order property after an insertion.	Restores the heap-order property after removing the root.
When	After a new key is inserted at the last position of the heap.	After removeMin, when the last key is moved to the root.
How	Swap the key with its parent, moving upward, until it is in place or reaches the root.	Swap the key with its smaller child, moving downward, until it is in place or reaches a leaf.
Time	`O(log n)`	`O(log n)`

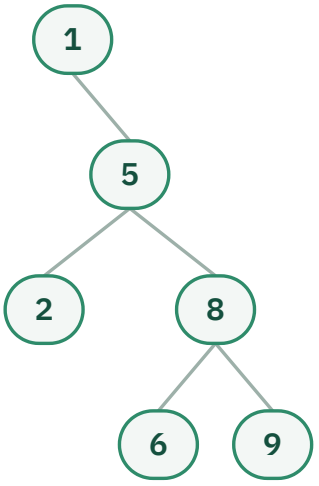
The time is `O(log n)` because the height of a heap is `log n`, and the key moves one level per swap.

b) A hash function is "good" when it disperses the keys over the cells in an apparently random, even way so that collisions are few, and when it is fast to compute.

03 Solution Q4: trees

a Remove node 3

Node 3 has two children (2 and 8). We replace it with the smallest key of its right subtree, which is 5 (the inorder successor), and remove the original node 5.



b Linked structure

Each node of the tree is represented by an object that stores three things:

- The element.
- A reference to the parent node.
- A sequence of references to the children nodes.

```
R  :  parent = ø     children = [r1, B, r2]
```

```
B  :  parent = R     children = [b1, D, b2]
```

```
D  :  parent = B     children = [d2, d3]
```

```
r1, r2, b1, b2, d2, d3 :  children = [ ]
```

04 Solution Q5: collision handling

Method	Explanation
Chaining	Each cell points to a linked list of the elements mapped there. It is simple, but it requires additional memory outside the table.
Linear probing	Open addressing: the colliding item is placed in the next available cell, circularly. Its weakness is that items lump together, which lengthens the probe sequences.
Double hashing	Open addressing: we use a second function `d(k)` and try the cells `(i + j·d(k)) mod N`. `d(k) ≠ 0` must hold and `N` must be a prime.

05 Solution Q6: counting operations

Line	# operations
currentMax ← A[0]	2
Temp ← A[0]	2
for i ← 1 to n − 1 do	1 + n
if A[i] > currentMax then	2(n − 1)
currentMax ← A[i]	2(n − 1)
{ increment counter i }	2(n − 1)
return currentMax	1
Total	**7n**

$$2 + 2 + (1 + n) + 2(n - 1) + 2(n - 1) + 2(n - 1) + 1 \;=\; 7n$$